https://hjfy.top/arxiv/2010.11929



# 一张图片值 16 乘 16 个词：Transformer 在大规模图像识别中的应用

Alexey Dosovitskiy*,†, Lucas Beyer*, Alexander Kolesnikov*, Dirk Weissenborn*,
Xiaohua Zhai*, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby*,†
*equal technical contribution, †equal advising
Google Research, Brain Team
{adosovitskiy, neilhoulsby}@google.com

## Abstract

尽管 Transformer 架构已成为自然语言处理任务的事实标准，但其在计算机视觉中的应用仍然有限。在视觉领域，注意力机制要么与卷积网络结合使用，要么用于替换卷积网络的某些组件，同时保持其整体结构不变。我们证明了对 CNN 的依赖并不是必要的，直接应用于图像块序列的纯 Transformer 在图像分类任务中可以表现出色。当在大量数据上预训练并转移到多个中等或小型图像识别基准（如 ImageNet、CIFAR-100、VTAB 等）时，Vision Transformer (ViT) 相比最先进的卷积网络取得了优异的结果，同时所需的计算资源显著减少。[1]

## 1 引言

自注意力-based 架构，尤其是 Transformer (Vaswani et al., 2017)，在自然语言处理（NLP）中已成为首选模型。主要方法是先在大规模文本语料库上进行预训练，然后在较小的任务特定数据集上微调 (Devlin et al., 2019)。由于 Transformer 在计算效率和可扩展性方面的优势，现在可以训练具有超过 100B 参数的前所未有的大模型 (Brown et al., 2020; Lepikhin et al., 2020)。随着模型和数据集的增长，性能饱和的迹象仍未显现。

在计算机视觉中，然而，卷积架构仍然占据主导地位 (LeCun et al., 1989; Krizhevsky et al., 2012; He et al., 2016)。受到自然语言处理成功的启发，多项工作尝试将类似于 CNN 的架构与自注意力相结合 (Wang et al., 2018; Carion et al., 2020)，有些甚至完全用自注意力取代了卷积 (Ramachandran et al., 2019; Wang et al., 2020a)。尽管这些后者的模型理论上效率更高，但由于使用了专门的注意力模式，在现代硬件加速器上尚未被有效扩展。因此，在大规模图像识别中，经典的 ResNet 类似架构仍然是最先进的 (Mahajan et al., 2018; Xie et al., 2020; Kolesnikov et al., 2020)。

受 Transformer 在自然语言处理（NLP）中缩放成功经验的启发，我们尝试直接将标准 Transformer 应用于图像，并进行尽可能少的修改。为此，我们将图像分割成小块，并将这些小块的线性嵌入序列作为输入提供给 Transformer。图像小块的处理方式与 NLP 应用中的词元（单词）相同。我们以监督的方式训练模型进行图像分类。

[1] 微调代码和预训练的模型可以在 https://github.com/google-research/vision_transformer 处获得。

当在中等规模的数据集（如 ImageNet ）上训练且没有强大的正则化时，这些模型的准确率仅比同等大小的 ResNets 低几个百分点，结果看似平平。这似乎令人沮丧的结果是可以预料的：Transformer 缺乏 CNN 所固有的某些归纳偏置，例如变换等变性和局部性，因此在数据量不足时无法很好地泛化。

然而，如果模型在更大的数据集（14M-300M 张图像）上进行训练，情况会发生变化。我们发现，大规模训练优于归纳偏置。我们的 Vision Transformer (ViT) 在足够大的规模下预训练并迁移到数据点较少的任务时，取得了出色的结果。当在公开的 ImageNet-21k 数据集或内部的 JFT-300M 数据集上预训练时，ViT 在多个图像识别基准上接近或超过了最先进的方法。特别是，最佳模型在 ImageNet 上达到了 88.55% 的准确率，在 ImageNet-ReaL 上达到了 90.72% 的准确率，在 CIFAR-100 上达到了 94.55% 的准确率，并在 VTAB 的 19 项任务集合上达到了 77.63% 的准确率。

## 2 相关工作

变换器由 Vaswani et al. (2017) 为机器翻译提出，此后在许多 NLP 任务中成为最先进的方法。基于变换器的大模型通常在大规模语料库上进行预训练，然后针对具体任务进行微调：BERT (Devlin et al., 2019) 使用去噪自监督预训练任务，而 GPT 系列工作则使用语言建模作为其预训练任务 (Radford et al., 2018; 2019; Brown et al., 2020)。

将每个像素对所有其他像素应用自注意力在图像上会导致像素数量的二次成本，这在实际输入大小下不可扩展。因此，在图像处理的背景下应用 Transformer 时，过去尝试了几种近似方法。Parmar et al. (2018) 仅对每个查询像素的局部邻域应用自注意力，而不是全局应用。这种局部多头点积自注意力块可以完全替代卷积 (Hu et al., 2019; Ramachandran et al., 2019; Zhao et al., 2020)。在另一条研究线中，稀疏 Transformer (Child et al., 2019) 使用可扩展的全局自注意力近似方法来应用于图像。另一种扩展注意力的方法是以不同大小的块应用它 (Weissenborn et al., 2019)，在极端情况下仅沿单个轴应用 (Ho et al., 2019; Wang et al., 2020a)。许多这些专门设计的注意力架构在计算机视觉任务中表现出有希望的结果，但需要复杂的工程才能在硬件加速器上高效实现。

与我们的工作最相关的是 Cordonnier et al. (2020) 的模型，它从输入图像中提取大小为 $2 \times 2$ 的补丁，并在其上应用全自注意力。这个模型非常类似于 ViT，但我们的工作进一步证明了大规模预训练使普通的 Transformer 模型在性能上可以媲美（甚至优于）最先进的 CNN。此外，Cordonnier et al. (2020) 使用了较小的补丁大小 $2 \times 2$ 像素，这使得该模型只能应用于低分辨率图像，而我们可以处理中等分辨率的图像。

近年来，将卷积神经网络（CNNs）与自注意力的多种形式相结合也引起了极大的兴趣，例如通过增强用于图像分类 (Bello et al., 2019) 的特征图 (Bello et al., 2019)，或者通过使用自注意力进一步处理 CNN 的输出，例如用于目标检测 (Hu et al., 2018; Carion et al., 2020)、视频处理 (Wang et al., 2018; Sun et al., 2019)、图像分类 (Wu et al., 2020)、无监督对象发现 (Locatello et al., 2020) 或统一的文本-视觉任务 (Chen et al., 2020c; Lu et al., 2019; Li et al., 2019)。

另一个最近相关的模型是图像 GPT（iGPT）(Chen et al., 2020a)，它通过降低图像分辨率和颜色空间后，将 Transformer 应用于图像像素。该模型以无监督的方式作为生成式模型进行训练，得到的表示可以进一步微调或线性探测以实现分类性能，在 ImageNet 上达到了 72% 的最高准确率。

我们的工作增加了越来越多的论文，这些论文探索了比标准 ImageNet 数据集更大规模的图像识别。使用额外的数据源可以在标准基准上实现最先进的结果 (Mahajan et al., 2018; Touvron et al., 2019; Xie et al., 2020)。此外，Sun et al. (2017) 研究了 CNN 性能随数据集大小的变化，而 Kolesnikov et al. (2020); Djolonga et al. (2020) 对从大规模数据集（如 ImageNet-21k 和 JFT-300M）进行 CNN 迁移学习进行了实证探索。我们同样关注这两个后两种数据集，但训练的是 Transformer 而不是之前工作中使用的基于 ResNet 的模型。

## 3 方法

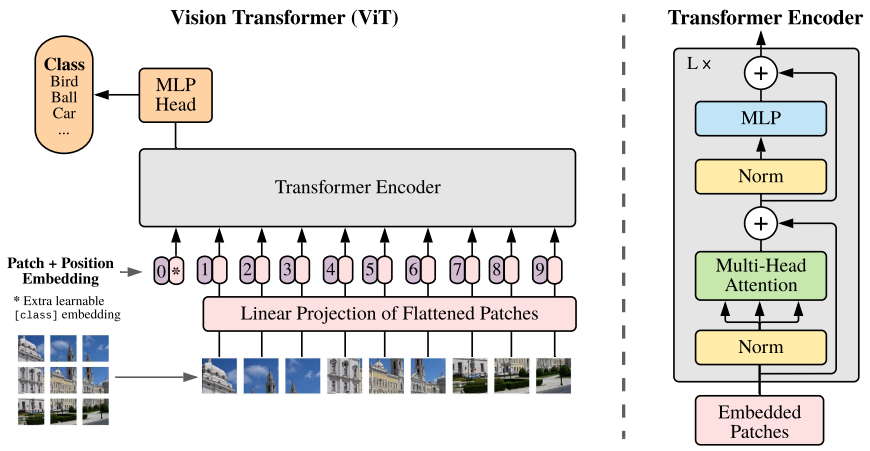



图 1: 模型概述。我们将图像分割成固定大小的块，对每个块进行线性嵌入，添加位置嵌入，然后将得到的向量序列输入到标准的 Transformer 编码器中。为了执行分类任务，我们使用了在序列中添加一个可学习的“分类标记”的标准方法。Transformer 编码器的插图灵感来源于 Vaswani et al. (2017)。

在模型设计中，我们尽可能地遵循原始的 Transformer (Vaswani et al., 2017)。这种刻意简单的设置有一个优势，即可扩展的 NLP Transformer 架构——以及它们的有效实现——几乎可以拿来即用。

### 3.1 Vision Transformer (ViT)

一个模型的概述如图 1 所示。标准 Transformer 接收一个 1D 的词元嵌入序列作为输入。为了处理 2D 图像，我们将图像 $\mathrm{x} \in \mathbb{R}^{H \times W \times C}$ 重塑为一系列展平的 2D 补丁 $\mathrm{x}_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$，其中 $(H, W)$ 是原始图像的分辨率，$C$ 是通道数，$(P, P)$ 是每个图像补丁的分辨率，$N = HW/P^2$ 是结果的补丁数量，这也作为 Transformer 的有效输入序列长度。Transformer 在所有层中使用恒定的潜在向量大小 $D$ ，因此我们展平补丁并使用可训练的线性投影将其映射到 $D$ 维度（方程 1）。我们将该投影的输出称为补丁嵌入。

类似于 BERT 的[class] 词元，我们在嵌入的补丁序列（$\mathrm{z}_0^0 = \mathrm{x}_{\mathrm{class}}$ ）前添加一个可学习的嵌入，Transformer 编码器输出处的状态（$\mathrm{z}_L^0$ ）作为图像表示 y (Eq. 4)。在预训练和微调过程中，都在 $\mathrm{z}_L^0$ 上附加了一个分类头。分类头在预训练时通过一个具有一个隐藏层的 MLP 实现，在微调时通过一个单一线性层实现。

将位置嵌入添加到块嵌入中以保留位置信息。由于我们没有观察到使用更高级的 2D 意识位置嵌入有显著的性能提升（附录 D.4），因此我们使用标准的可学习 1D 位置嵌入。得到的嵌入向量序列作为编码器的输入。

编码器 (Vaswani et al., 2017) 由多头自注意力（MSA，见附录 A）和 MLP 块（公式 2，3）交替组成。在每个块之前应用层归一化（LN），并在每个块之后应用残差连接 (Wang et al., 2019; Baevski & Auli, 2019)。MLP 包含两层，并具有 GELU 非线性。

$$\mathrm{z}_0 = [\mathrm{x}_{\text{class}}; \mathrm{x}_p^1\mathrm{E}; \mathrm{x}_p^2\mathrm{E}; \cdots; \mathrm{x}_p^N\mathrm{E}] + \mathrm{E}_{pos}, \qquad \mathrm{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathrm{E}_{pos} \in \mathbb{R}^{(N+1) \times D} \tag{1}$$

$$\mathrm{z}'_\ell = \mathrm{MSA}(\mathrm{LN}(\mathrm{z}_{\ell-1})) + \mathrm{z}_{\ell-1}, \qquad \ell = 1 \ldots L \tag{2}$$

$$\mathrm{z}_\ell = \mathrm{MLP}(\mathrm{LN}(\mathrm{z}'_\ell)) + \mathrm{z}'_\ell, \qquad \ell = 1 \ldots L \tag{3}$$

$$\mathrm{y} = \mathrm{LN}(\mathrm{z}_L^0) \tag{4}$$

**归纳偏置**。 我们注意到，Vision Transformer 比 CNN 具有更少的图像特定的归纳偏置。在 CNN 中，局部性、二维邻域结构和平移等变性被融入到整个模型的每一层中。在 ViT 中，只有 MLP 层是局部且平移等变的，而自注意力层是全局的。二维邻域结构仅在模型开始时通过将图像切割成块以及在微调阶段用于调整不同分辨率图像的位置嵌入（如下所述）时使用。除此之外，在初始化时的位置嵌入不携带任何关于块的二维位置及其所有空间关系的信息，这些关系必须从头开始学习。

**混合架构**。 作为原始图像块的替代方案，输入序列可以由 CNN 的特征图组成 (LeCun et al., 1989)。在该混合模型中，应用补丁嵌入投影 E（Eq. 1）到从 CNN 特征图提取的补丁上。作为一种特殊情况，补丁的空间大小可以是 1x1，这意味着通过简单地展平特征图的空间维度并投影到 Transformer 维度来获得输入序列。如上所述，分类输入嵌入和位置嵌入被添加。

### 3.2 微调与更高分辨率

通常，我们在大型数据集上对 ViT 进行预训练，并微调到（较小的）下游任务。为此，我们移除预训练的预测头，并附加一个零初始化的 $D \times K$ 前馈层，其中 $K$ 是下游类别的数量。通常，在比预训练更高的分辨率下微调是有益的 (Touvron et al., 2019; Kolesnikov et al., 2020)。当输入更高分辨率的图像时，我们保持 patch 大小不变，这会导致有效的序列长度变大。Vision Transformer 可以处理任意长度的序列（受内存约束限制），然而，预训练的位置嵌入可能不再有意义。因此，我们根据原始图像中的位置对预训练的位置嵌入进行二维插值。请注意，这种分辨率调整和 patch 提取是唯一手动将关于图像二维结构的归纳偏置注入到 Vision Transformer 中的地方。

## 4 实验

我们评估了 ResNet、Vision Transformer（ViT ）和混合模型的表示学习能力。为了理解每个模型的数据需求，我们在不同大小的数据集上进行预训练，并评估许多基准任务。在考虑预训练模型的计算成本时，ViT 表现出色，在大多数识别基准上达到了最先进的性能，且预训练成本更低。最后，我们使用自监督进行一个小实验，表明自监督的 ViT 对未来有希望的发展前景。

### 4.1 设置

**数据集**。为了探索模型的可扩展性，我们使用了 ILSVRC-2012 ImageNet 数据集（包含 1k 类和 130 万张图像，后续简称 ImageNet ）、其超集 ImageNet-21k（包含 21k 类和 1400 万张

| Model | Layers | Hidden size $D$ | MLP size | Heads | Params |
|---|---|---|---|---|---|
| ViT-Base | 12 | 768 | 3072 | 12 | 86M |
| ViT-Large | 24 | 1024 | 4096 | 16 | 307M |
| ViT-Huge | 32 | 1280 | 5120 | 16 | 632M |

表 1: Vision Transformer 模型变体的细节。

图像 (Deng et al., 2009)）以及 JFT (Sun et al., 2017)（包含 18k 类和 3.03 亿高分辨率图像）。我们按照 Kolesnikov et al. (2020) 的方法，针对下游任务的测试集去重预训练数据集。我们将这些数据集上训练的模型迁移到多个基准任务：在原始验证标签和清理后的 ReaL 标签上的 ImageNet (Beyer et al., 2020)、CIFAR-10/100 (Krizhevsky, 2009)、Oxford-IIIT Pets (Parkhi et al., 2012) 和 Oxford Flowers-102 (Nilsback & Zisserman, 2008)。对于这些数据集，预处理遵循 Kolesnikov et al. (2020)。

我们还在 19 项任务的 VTAB 分类套件上进行了评估 (Zhai et al., 2019b)。VTAB 评估了低数据迁移至多样化任务的能力，每项任务使用 1,000 个训练样本。这些任务分为三组：*自然* – 像上述任务、宠物、CIFAR 等。*专业化* – 医学和卫星图像等，*结构化* – 需要几何理解的任务，如定位。

**模型变体**。我们基于用于 BERT (Devlin et al., 2019) 的配置来设置 ViT ，如表 1 中总结的那样。“Base” 和 “Large” 模型直接采用自 BERT，我们还添加了更大的 “Huge” 模型。接下来，我们将使用简短的符号来表示模型大小和输入 patch 的大小：例如，ViT-L/16 表示具有 $16 \times 16$ 输入 patch 大小的 “Large” 变体。注意，Transformer 的序列长度与 patch 大小的平方成反比，因此 patch 大小较小的模型在计算上更昂贵。

对于基准 CNNs，我们使用 ResNet (He et al., 2016)，但用组规范化 (Wu & He, 2018) 替换了批量规范化层 (Ioffe & Szegedy, 2015)，并使用了标准化卷积 (Qiao et al., 2019)。这些修改改善了迁移 (Kolesnikov et al., 2020)，我们将修改后的模型记为 “ResNet (BiT)”。对于混合模型，我们将中间特征图输入到 ViT 中，patch 大小为一个 “像素”。为了试验不同的序列长度，我们可以选择：(i) 使用常规 ResNet50 的第四阶段输出，或者 (ii) 移除第四阶段，在第三阶段添加相同数量的层（保持总层数不变），并使用扩展后的第三阶段输出。选项 (ii) 会将序列长度增加 4 倍，并且需要更昂贵的 ViT 模型。

**训练 & 微调**。我们使用 Adam (Kingma & Ba, 2015) 训练所有模型，包括 ResNets，并设置 $\beta_1 = 0.9$ ，$\beta_2 = 0.999$ ，批量大小为 4096，并应用了高权重衰减 0.1 ，我们发现这对所有模型的迁移非常有用（附录 D.1 显示，在我们的设定中，Adam 相对于常见的 SGD 做法对 ResNets 稍微更有效）。我们使用线性学习率预热和衰减，详见附录 B.1 。对于微调，我们使用带有动量的 SGD，批量大小为 512，适用于所有模型，详见附录 B.1.1 。对于表 2 中的 ImageNet 结果，我们在更高分辨率下进行了微调：512 对于 ViT-L/16 和 518 对于 ViT-H/14，并且还使用了 Polyak & Juditsky (1992) 平均值，因子为 0.9999 (Ramachandran et al., 2019; Wang et al., 2020b)。

**指标**。我们通过 few-shot 或微调准确率来报告下游数据集上的结果。微调准确率反映了每个模型在各自数据集上进行微调后的性能。Few-shot 准确率是通过求解一个正则化最小二乘回归问题得到的，该问题将训练图像子集的（固定）表示映射到 $\{-1, 1\}^K$ 目标向量。这种公式化方法使我们能够以封闭形式得到确切的解。虽然我们主要关注微调性能，但有时我们会使用线性 few-shot 准确率进行快速的即时评估，因为在这些情况下进行微调的成本太高。

| | Ours-JFT (ViT-H/14) | Ours-JFT (ViT-L/16) | Ours-I21k (ViT-L/16) | BiT-L (ResNet152x4) | Noisy Student (EfficientNet-L2) |
|---|---|---|---|---|---|
| ImageNet | $88.55 \pm 0.04$ | $87.76 \pm 0.03$ | $85.30 \pm 0.02$ | $87.54 \pm 0.02$ | 88.4/88.5* |
| ImageNet ReaL | $90.72 \pm 0.05$ | $90.54 \pm 0.03$ | $88.62 \pm 0.05$ | 90.54 | 90.55 |
| CIFAR-10 | $99.50 \pm 0.06$ | $99.42 \pm 0.03$ | $99.15 \pm 0.03$ | $99.37 \pm 0.06$ | – |
| CIFAR-100 | $94.55 \pm 0.04$ | $93.90 \pm 0.05$ | $93.25 \pm 0.05$ | $93.51 \pm 0.08$ | – |
| Oxford-IIIT Pets | $97.56 \pm 0.03$ | $97.32 \pm 0.11$ | $94.67 \pm 0.15$ | $96.62 \pm 0.23$ | – |
| Oxford Flowers-102 | $99.68 \pm 0.02$ | $99.74 \pm 0.00$ | $99.61 \pm 0.02$ | $99.63 \pm 0.03$ | – |
| VTAB (19 tasks) | $77.63 \pm 0.23$ | $76.28 \pm 0.46$ | $72.72 \pm 0.21$ | $76.29 \pm 1.70$ | – |
| TPUv3-core-days | 2.5k | 0.68k | 0.23k | 9.9k | 12.3k |

表 2: 与主流图像分类基准上的最新技术进行比较。我们报告了平均准确率及其标准差，结果是通过三次微调运行的平均值。在 JFT-300M 数据集上预训练的 Vision Transformer 模型在所有数据集上都优于基于 ResNet 的基准模型，同时所需的计算资源更少。在较小的公开 ImageNet-21k 数据集上预训练的 ViT 表现也很出色。* 略有改进的 88.5% 结果见 Touvron et al. (2020)。

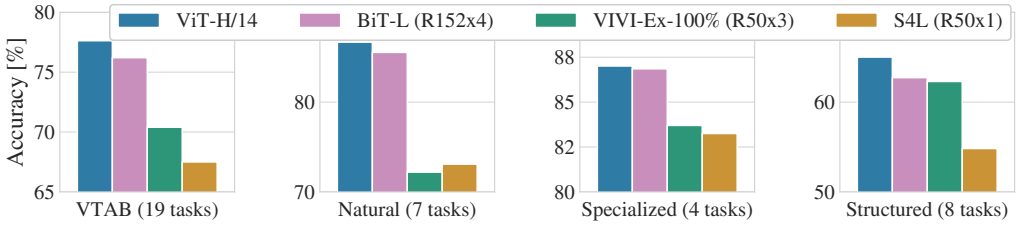



图 2: 自然、专业化和结构化任务组中 VTAB 性能的分解。

### 4.2 与最先进的技术相比

我们首先将我们最大的模型 – ViT-H/14 和 ViT-L/16 – 与文献中的最先进 CNN 进行比较。第一个比较点是 Big Transfer (BiT) (Kolesnikov et al., 2020)，它使用大型 ResNets 执行监督迁移学习。第二个是 Noisy Student (Xie et al., 2020)，它是一种通过在移除标签的 ImageNet 和 JFT-300M 上使用半监督学习训练的大 EfficientNet。目前，Noisy Student 在 ImageNet 上处于最先进水平，而 BiT-L 在其他报告的数据集上处于最先进水平。所有模型都在 TPUv3 硬件上进行了训练，我们报告了每个模型预训练所需的 TPUv3-core 天数，即用于训练的 TPU v3 核心数（每芯片 2 个）乘以训练时间（天数）。

表 2 显示了结果。较小的 ViT-L/16 模型在 JFT-300M 上预训练，在所有任务上都优于在相同数据集上预训练的 BiT-L，同时需要更少的计算资源来训练。较大的模型，ViT-H/14，进一步提高了性能，特别是在更具挑战性的数据集 – ImageNet 、CIFAR-100 和 VTAB 套件上。有趣的是，这个模型仍然比先前的最先进的方法需要更少的计算资源进行预训练。然而，我们注意到预训练效率不仅可能受到架构选择的影响，还可能受到其他参数的影响，例如训练计划、优化器、权重衰减等。我们在第 4.4 节提供了不同架构在性能与计算之间的受控研究。最后，使用公共 ImageNet-21k 数据集预训练的 ViT-L/16 模型在大多数数据集上表现良好，同时需要更少的资源进行预训练：它可以在大约 30 天内在标准云 TPUv3 的 8 核上完成训练。

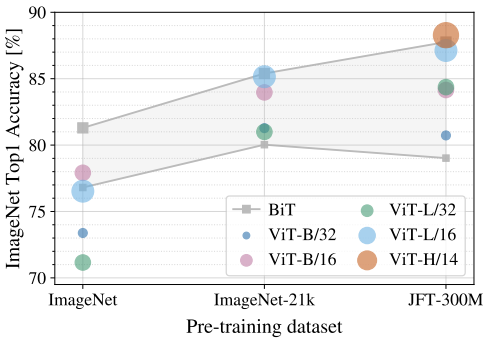



图 3: 迁移到 ImageNet 。当在小数据集上进行预训练时，大型 ViT 模型的表现不如 BiT ResNets（阴影区域所示）；然而，当在大数据集上进行预训练时，它们表现出色。同样地，随着数据集的增大，更大的 ViT 变体超过了较小的变体。

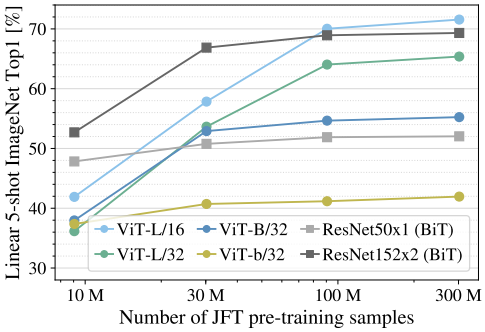



图 4: 线性的 few-shot 在 ImageNet 上的评估相对于预训练的 Size。当预训练数据集较小时，ResNets 表现更好，但比我们的方法 ViT 更快达到性能瓶颈，ViT 在较大的预训练数据集上表现更好。ViT-b 是将 ViT-B 的所有隐藏 Dimension 减半得到的。

图 2 将 VTAB 任务分解为其各自的组，并与该基准上的先前 SOTA 方法进行比较：BiT，VIVI – 在 ImageNet 和 Youtube (Tschannen et al., 2020) 上共同训练的 ResNet，以及 S4L – 在 ImageNet (Zhai et al., 2019a) 上的监督加半监督学习。ViT-H/14 在自然和结构化任务上优于 BiT-R152x4 和其他方法。在专业化任务中，顶级两个模型的性能相似。

### 4.3 预训练数据要求

我们的方法在预训练于大规模 JFT-300M 数据集时表现良好。相比于 ResNet 具有更少的视觉归纳偏置，数据集 Size 有多重要？我们进行了两组实验。

首先，我们在数据集大小不断增加的情况下对 ViT 模型进行预训练：ImageNet 、ImageNet-21k 和 JFT-300M。为了提升在较小数据集上的性能，我们优化了三个基本的正则化参数——权重衰减、暂退法 (Dropout) 和标签平滑。图 3 显示了微调到 ImageNet 后的结果（其他数据集的结果见表 5）。[2]. 当在最小的数据集，ImageNet ，上进行预训练时，尽管有（适度的）正则化，ViT-Large 模型的表现仍不如 ViT-Base 模型。使用 ImageNet-21k 进行预训练后，它们的表现相似。只有在 JFT-300M 上进行预训练时，我们才能看到更大模型的全部优势。图 3 还展示了不同大小的 BiT 模型所覆盖的性能区域。BiT CNN 在 ImageNet 上的表现优于 ViT ，但在更大的数据集上，ViT 超过了 BiT。

其次，我们在 9M、30M 和 90M 的随机子集以及完整的 JFT-300M 数据集上训练我们的模型。我们不对较小的子集进行额外的正则化，并在所有情景中使用相同的超参数。这样，我们可以评估模型的内在属性，而不是正则化的效应。然而，我们确实使用了早停法，并报告训练期间达到的最佳验证准确率。为了节省计算资源，我们报告 few-shot 线性准确率而不是完全微调的准确率。图 4 包含结果。Vision Transformer 在计算成本相当的情况下比小数据集上的 ResNet 更容易过拟合。例如，ViT-B/32 比 ResNet50 稍快；它在 9M 子集上的表现较差，但在 90M+子集上表现更好。对于 ResNet152x2 和 ViT-L/16 来说也是如此。这一结果强化了这样的直觉：卷积归纳偏置对小数据集有用，但对于较大的数据集，直接从数据中学习相关模式就足够了，甚至是有益的。

[2]请注意，ImageNet 预训练模型也进行了微调，但同样是在 ImageNet 上进行的。这是因为微调期间分辨率的提高提升了性能。

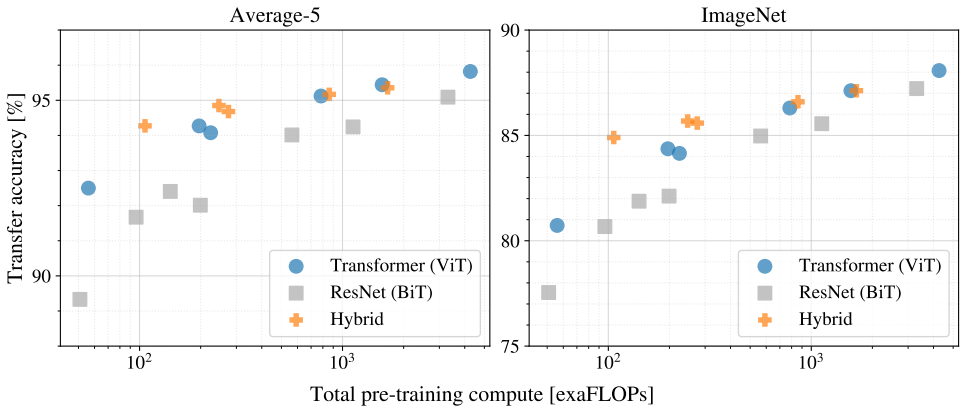



图 5: 性能与不同架构的预训练计算量的关系：Vision Transformer 、ResNet 和混合模型。在相同的计算预算下，Vision Transformer 通常优于 ResNet。对于较小的模型大小，混合模型比纯 Transformer 更好，但对于较大的模型，差距会消失。

总体而言，ImageNet 上的 few-shot 结果（图 4）以及 VTAB 上的低数据结果（表 2）在非常低的数据迁移方面似乎很有前景。对 ViT 的 few-shot 特性进行进一步分析是未来工作的令人兴奋的方向。

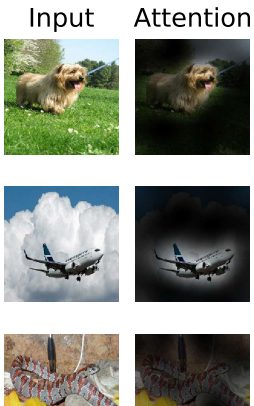
### 4.4 缩放研究

我们通过从 JFT-300M 进行迁移性能评估，对不同模型执行受控的缩放研究。在此情景下，数据大小不会限制模型的表现，我们评估每个模型相对于预训练成本的表现。模型集包括：7 个 ResNet，R50x1, R50x2 R101x1, R152x1, R152x2，预训练 7 轮，加上 R152x2 和 R200x3 预训练 14 轮；6 个 Vision Transformer ，ViT-B/32, B/16, L/32, L/16，预训练 7 轮，加上 L/16 和 H/14 预训练 14 轮；以及 5 个混合模型，R50+ViT-B/32, B/16, L/32, L/16 预训练 7 轮，加上 R50+ViT-L/16 预训练 14 轮（对于混合模型，模型名称末尾的数字不代表 patch 大小，而是代表 ResNet 主干中的总下采样比）。

图 5 包含了转移性能与总预训练计算成本的对比（有关计算成本的详细信息请参见附录 D.5）。每种模型的详细结果在附录中的表 6 中提供。可以观察到几个模式。首先，Vision Transformer 在性能/计算权衡方面优于 ResNets。ViT 使用大约 $2-4\times$ 的计算成本就可以达到相同的性能（5 个数据集的平均值）。其次，在较小的计算预算下，混合模型略优于 ViT ，但在更大规模的模型中，这种差异消失了。这个结果有些出人意料，因为人们可能会认为卷积局部特征处理会在任何规模下帮助 ViT 。第三，Vision Transformer 在尝试的范围内似乎没有饱和，这激励了未来的扩展工作。

### 4.5 检查 Vision Transformer

要开始理解 Vision Transformer 如何处理图像数据，我们分析了其内部表示。Vision Transformer 的第一层将展平的补丁线性投影到一个低维空间中（Eq. 1）。图 7（左）显示了学成嵌入滤波器的主要成分。这些成分类似于每个补丁内精细结构的低维表示的合理基函数。




在投影之后，一个学成的位置嵌入被添加到补丁表示中。图 7（中心）显示，模型学会了在位置嵌入的相似度中对图像内的

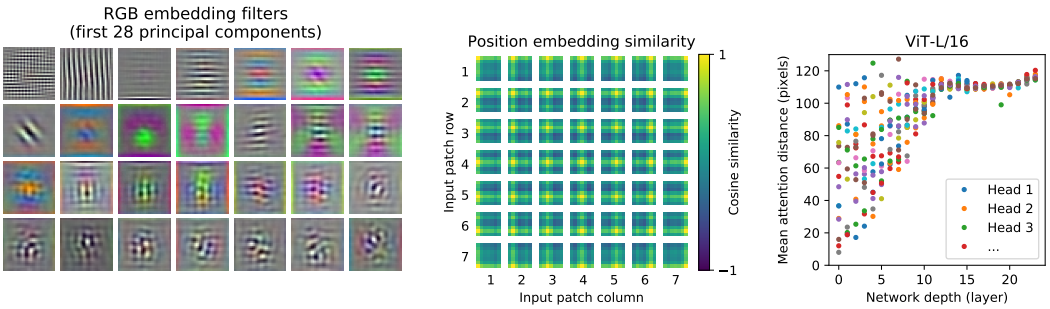



图 7: **左**: ViT-L/32 的初始线性嵌入的过滤器。**中**: ViT-L/32 的位置嵌入的相似度。瓷砖显示具有指定行和列的补丁的位置嵌入与所有其他补丁的位置嵌入之间的余弦相似度。**右**: 头部和网络深度所关注区域的大小。每个点显示一层中 16 个头部之一在图像上的平均注意距离。有关详细信息，请参见附录 D.7。

距离进行编码，即较近的补丁倾向于具有更相似的位置嵌入。此外，行-列结构出现了；同一行/列中的补丁具有相似的嵌入。最后，在较大的网格中有时会出现正弦结构（附录 D）。位置嵌入学会表示 2D 图像拓扑结构，这就是为什么手工制作的 2D 感知嵌入变体没有带来改进的原因（附录 D.4）。

自注意力允许 ViT 在最底层整合整个图像的信息。我们研究了网络在多大程度上利用了这种能力。具体来说，我们根据注意力权重计算图像空间中信息被整合的平均距离（图 7，右）。这个 “注意力距离” 类似于 CNN 中的感受野大小。我们发现一些注意力头在最低层就已经关注到大部分图像，表明模型确实利用了整合全局信息的能力。其他注意力头在低层具有始终较小的注意力距离。这种高度局部化的注意力在应用 ResNet 后再使用 Transformer 的混合模型中表现得不那么明显（图 7，右），这表明它可能在功能上类似于 CNN 中的早期卷积层。此外，注意力距离随着网络深度增加而增大。总体而言，我们发现模型关注的是对分类语义相关的图像区域（图 6）。

### 4.6 自监督

Transformer 在自然语言处理（NLP）任务中表现出了令人印象深刻的性能。然而，它们的成功很大程度上不仅源于其出色的可扩展性，还源于大规模的自监督预训练 (Devlin et al., 2019; Radford et al., 2018)。我们还对用于自监督的 *掩码块预测* 进行了初步探索，模仿了 BERT 中使用的掩码语言模型化任务。通过自监督预训练，我们较小的 ViT-B/16 模型在 ImageNet 上达到了 79.9% 的准确率，与从头开始训练相比显著提高了 2%，但仍比监督预训练低 4%。附录 B.1.2 包含更多详细信息。我们将对比预训练 (Chen et al., 2020b; He et al., 2020; Bachman et al., 2019; Hénaff et al., 2020) 的探索留待未来工作。

## 5 结论

我们探索了将 Transformer 直接应用于图像识别的方法。与之前在计算机视觉中使用自注意力的工作不同，除了最初的图像块提取步骤外，我们没有向架构引入特定于图像的归纳偏

置。相反，我们将图像解释为一系列图像块，并通过自然语言处理中常用的 Transformer 编码器对其进行处理。当与大规模数据集上的预训练结合时，这种简单但可扩展的策略表现得非常好。因此，Vision Transformer 在许多图像分类数据集上达到了或超过了最先进的水平，同时预训练的成本相对较低。

虽然这些初步结果令人鼓舞，但仍有许多挑战。其中之一是将 ViT 应用到其他计算机视觉任务上，例如检测和分割。我们的结果，连同 Carion et al. (2020) 中的结果，表明了这种方法的潜力。另一个挑战是继续探索自监督预训练方法。我们的初始实验显示自监督预训练带来了改进，但自监督与大规模监督预训练之间仍存在很大差距。最后，进一步扩展 ViT 可能会带来性能的提升。

## 致谢

这项工作在柏林、苏黎世和阿姆斯特丹完成。我们感谢谷歌的许多同事的帮助，特别是 Andreas Steiner 在基础设施和代码开源方面的关键帮助；Joan Puigcerver 和 Maxim Neumann 在大规模训练基础设施方面的帮助；Dmitry Lepikhin、Aravindh Mahendran、Daniel Keysers、Mario Lučić、Noam Shazeer、Ashish Vaswani 和 Colin Raffel 提供了有益的讨论。

## 参考文献

Samira Abnar and Willem Zuidema. Quantifying attention flow in transformers. In ACL, 2020.

Philip Bachman, R Devon Hjelm, and William Buchwalter. Learning representations by maximizing mutual information across views. In NeurIPS, 2019.

Alexei Baevski and Michael Auli. Adaptive input representations for neural language modeling. In ICLR, 2019.

I. Bello, B. Zoph, Q. Le, A. Vaswani, and J. Shlens. Attention augmented convolutional networks. In ICCV, 2019.

Lucas Beyer, Olivier J. Hénaff, Alexander Kolesnikov, Xiaohua Zhai, and Aäron van den Oord. Are we done with imagenet? arXiv, 2020.

Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. arXiv, 2020.

Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In ECCV, 2020.

Mark Chen, Alec Radford, Rewon Child, Jeff Wu, and Heewoo Jun. Generative pretraining from pixels. In ICML, 2020a.

Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey E. Hinton. A simple framework for contrastive learning of visual representations. In ICML, 2020b.

Yen-Chun Chen, Linjie Li, Licheng Yu, Ahmed El Kholy, Faisal Ahmed, Zhe Gan, Yu Cheng, and Jingjing Liu. UNITER: UNiversal Image-TExt Representation Learning. In ECCV, 2020c.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. arXiv, 2019.

Jean-Baptiste Cordonnier, Andreas Loukas, and Martin Jaggi. On the relationship between self-attention and convolutional layers. In ICLR, 2020.

J. Deng, W. Dong, R. Socher, L. Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In CVPR, 2009.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In NAACL, 2019.

Josip Djolonga, Jessica Yung, Michael Tschannen, Rob Romijnders, Lucas Beyer, Alexander Kolesnikov, Joan Puigcerver, Matthias Minderer, Alexander D'Amour, Dan Moldovan, Sylvan Gelly, Neil Houlsby, Xiaohua Zhai, and Mario Lucic. On robustness and transferability of convolutional neural networks. arXiv, 2020.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In CVPR, 2016.

Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning. In CVPR, 2020.

Jonathan Ho, Nal Kalchbrenner, Dirk Weissenborn, and Tim Salimans. Axial attention in multidimensional transformers. arXiv, 2019.

Han Hu, Jiayuan Gu, Zheng Zhang, Jifeng Dai, and Yichen Wei. Relation networks for object detection. In CVPR, 2018.

Han Hu, Zheng Zhang, Zhenda Xie, and Stephen Lin. Local relation networks for image recognition. In ICCV, 2019.

Zilong Huang, Xinggang Wang, Yunchao Wei, Lichao Huang, Humphrey Shi, Wenyu Liu, and Thomas S. Huang. Ccnet: Criss-cross attention for semantic segmentation. In ICCV, 2020.

Olivier J. Hénaff, Aravind Srinivas, Jeffrey De Fauw, Ali Razavi, Carl Doersch, S. M. Ali Eslami, and Aaron van den Oord. Data-efficient image recognition with contrastive predictive coding. In ICML, 2020.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. 2015.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In ICLR, 2015.

Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Joan Puigcerver, Jessica Yung, Sylvain Gelly, and Neil Houlsby. Big transfer (BiT): General visual representation learning. In ECCV, 2020.

Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, 2009.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In NIPS, 2012.

Y. LeCun, B. Boser, J. Denker, D. Henderson, R. Howard, W. Hubbard, and L. Jackel. Backpropagation applied to handwritten zip code recognition. Neural Computation, 1: 541–551, 1989.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. arXiv, 2020.

Liunian Harold Li, Mark Yatskar, Da Yin, Cho-Jui Hsieh, and Kai-Wei Chang. VisualBERT: A Simple and Performant Baseline for Vision and Language. In Arxiv, 2019.

Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. arXiv, 2020.

Jiasen Lu, Dhruv Batra, Devi Parikh, and Stefan Lee. ViLBERT: Pretraining Task-Agnostic Visiolinguistic Representations for Vision-and-Language Tasks. In NeurIPS. 2019.

Dhruv Mahajan, Ross Girshick, Vignesh Ramanathan, Kaiming He, Manohar Paluri, Yixuan Li, Ashwin Bharambe, and Laurens van der Maaten. Exploring the limits of weakly supervised pretraining. In ECCV, 2018.

M. Nilsback and A. Zisserman. Automated flower classification over a large number of classes. In ICVGIP, 2008.

Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman, and C. V. Jawahar. Cats and dogs. In CVPR, 2012.

Niki Parmar, Ashish Vaswani, Jakob Uszkoreit, Lukasz Kaiser, Noam Shazeer, Alexander Ku, and Dustin Tran. Image transformer. In ICML, 2018.

B. T. Polyak and A. B. Juditsky. Acceleration of stochastic approximation by averaging. SIAM Journal on Control and Optimization, 30(4):838–855, 1992. doi: 10.1137/0330046. URL https://doi.org/10.1137/0330046.

Siyuan Qiao, Huiyu Wang, Chenxi Liu, Wei Shen, and Alan Yuille. Weight standardization. arXiv preprint arXiv:1903.10520, 2019.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding with unsupervised learning. Technical Report, 2018.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical Report, 2019.

Prajit Ramachandran, Niki Parmar, Ashish Vaswani, Irwan Bello, Anselm Levskaya, and Jon Shlens. Stand-alone self-attention in vision models. In NeurIPS, 2019.

Chen Sun, Abhinav Shrivastava, Saurabh Singh, and Abhinav Gupta. Revisiting unreasonable effectiveness of data in deep learning era. In ICCV, 2017.

Chen Sun, Austin Myers, Carl Vondrick, Kevin Murphy, and Cordelia Schmid. Videobert: A joint model for video and language representation learning. In ICCV, 2019.

Hugo Touvron, Andrea Vedaldi, Matthijs Douze, and Herve Jegou. Fixing the train-test resolution discrepancy. In NeurIPS. 2019.

Hugo Touvron, Andrea Vedaldi, Matthijs Douze, and Herve Jegou. Fixing the train-test resolution discrepancy: Fixefficientnet. arXiv preprint arXiv:2003.08237, 2020.

Michael Tschannen, Josip Djolonga, Marvin Ritter, Aravindh Mahendran, Neil Houlsby, Sylvain Gelly, and Mario Lucic. Self-supervised learning of video-induced visual invariances. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2020.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In NIPS, 2017.

Huiyu Wang, Yukun Zhu, Bradley Green, Hartwig Adam, Alan Yuille, and Liang-Chieh Chen. Axial-deeplab: Stand-alone axial-attention for panoptic segmentation. In ECCV, 2020a.

Huiyu Wang, Yukun Zhu, Bradley Green, Hartwig Adam, Alan Yuille, and Liang-Chieh Chen. Axial-deeplab: Stand-alone axial-attention for panoptic segmentation. arXiv preprint arXiv:2003.07853, 2020b.

Qiang Wang, Bei Li, Tong Xiao, Jingbo Zhu, Changliang Li, Derek F. Wong, and Lidia S. Chao. Learning deep transformer models for machine translation. In ACL, 2019.

Xiaolong Wang, Ross Girshick, Abhinav Gupta, and Kaiming He. Non-local neural networks. In CVPR, 2018.

Dirk Weissenborn, Oscar Täckström, and Jakob Uszkoreit. Scaling autoregressive video models. In ICLR, 2019.

Bichen Wu, Chenfeng Xu, Xiaoliang Dai, Alvin Wan, Peizhao Zhang, Masayoshi Tomizuka, Kurt Keutzer, and Peter Vajda. Visual transformers: Token-based image representation and processing for computer vision. arxiv, 2020.

Yuxin Wu and Kaiming He. Group normalization. In ECCV, 2018.

Qizhe Xie, Minh-Thang Luong, Eduard Hovy, and Quoc V. Le. Self-training with noisy student improves imagenet classification. In CVPR, 2020.

Xiaohua Zhai, Avital Oliver, Alexander Kolesnikov, and Lucas Beyer. $S^4L$: Self-Supervised Semi-Supervised Learning. In ICCV, 2019a.

Xiaohua Zhai, Joan Puigcerver, Alexander Kolesnikov, Pierre Ruyssen, Carlos Riquelme, Mario Lucic, Josip Djolonga, Andre Susano Pinto, Maxim Neumann, Alexey Dosovitskiy, et al. A large-scale study of representation learning with the visual task adaptation benchmark. arXiv preprint arXiv:1910.04867, 2019b.

Hengshuang Zhao, Jiaya Jia, and Vladlen Koltun. Exploring self-attention for image recognition. In CVPR, 2020.

| Models | Dataset | Epochs | Base LR | LR decay | Weight decay | Dropout |
|---|---|---|---|---|---|---|
| ViT-B/{16,32} | JFT-300M | 7 | $8 \cdot 10^{-4}$ | linear | 0.1 | 0.0 |
| ViT-L/32 | JFT-300M | 7 | $6 \cdot 10^{-4}$ | linear | 0.1 | 0.0 |
| ViT-L/16 | JFT-300M | 7/14 | $4 \cdot 10^{-4}$ | linear | 0.1 | 0.0 |
| ViT-H/14 | JFT-300M | 14 | $3 \cdot 10^{-4}$ | linear | 0.1 | 0.0 |
| R50x{1,2} | JFT-300M | 7 | $10^{-3}$ | linear | 0.1 | 0.0 |
| R101x1 | JFT-300M | 7 | $8 \cdot 10^{-4}$ | linear | 0.1 | 0.0 |
| R152x{1,2} | JFT-300M | 7 | $6 \cdot 10^{-4}$ | linear | 0.1 | 0.0 |
| R50+ViT-B/{16,32} | JFT-300M | 7 | $8 \cdot 10^{-4}$ | linear | 0.1 | 0.0 |
| R50+ViT-L/32 | JFT-300M | 7 | $2 \cdot 10^{-4}$ | linear | 0.1 | 0.0 |
| R50+ViT-L/16 | JFT-300M | 7/14 | $4 \cdot 10^{-4}$ | linear | 0.1 | 0.0 |
| ViT-B/{16,32} | ImageNet-21k | 90 | $10^{-3}$ | linear | 0.03 | 0.1 |
| ViT-L/{16,32} | ImageNet-21k | 30/90 | $10^{-3}$ | linear | 0.03 | 0.1 |
| ViT-$*$ | ImageNet | 300 | $3 \cdot 10^{-3}$ | cosine | 0.3 | 0.1 |

表 3: 超参数用于训练。所有模型均以 4096 的批量大小和 10k 步的学习率预热进行训练。对于 ImageNet ，我们发现额外应用全局范数为 1 的梯度裁剪是有益的。训练分辨率为 224。

# 附录

## A 多头自注意力

标准 qkv 自注意力（SA, Vaswani et al. (2017)）是神经架构的流行构建块。对于输入序列 $\mathrm{z} \in \mathbb{R}^{N \times D}$ 中的每个元素，我们计算该序列中所有值 v 的加权和。注意权重 $A_{ij}$ 基于序列中两个元素之间的成对相似度及其各自的查询 $\mathrm{q}^i$ 和键 $\mathrm{k}^j$ 表示。

$$[\mathrm{q}, \mathrm{k}, \mathrm{v}] = \mathrm{z}\mathrm{U}_{qkv} \qquad \mathrm{U}_{qkv} \in \mathbb{R}^{D \times 3D_h}, \tag{5}$$

$$A = \mathrm{softmax}\left(\mathrm{q}\mathrm{k}^\top / \sqrt{D_h}\right) \qquad A \in \mathbb{R}^{N \times N}, \tag{6}$$

$$\mathrm{SA}(\mathrm{z}) = A\mathrm{v}\,. \tag{7}$$

多头自注意力（MSA）是 SA 的扩展，在其中我们并行运行 $k$ 个自注意力操作，称为“头”，并将它们的连结输出投影。为了在改变 $k$ 时保持计算和参数数量不变，通常将 $D_h$（公式 5）设置为 $D/k$。

$$\mathrm{MSA}(\mathrm{z}) = [\mathrm{SA}_1(z); \mathrm{SA}_2(z); \cdots; \mathrm{SA}_k(z)]\, \mathrm{U}_{msa} \qquad \mathrm{U}_{msa} \in \mathbb{R}^{k \cdot D_h \times D} \tag{8}$$

## B 实验细节

### B.1 训练

表 3 概述了我们不同模型的训练设置。我们发现，在 ImageNet 上从头开始训练模型时，强大的正则化是关键。当使用暂退法 (Dropout) 时，它在每个稠密层之后应用，但 qkv 投影以及在将位置信息添加到嵌入之前除外。混合模型的训练设置与其 ViT 对应模型完全相同。最后，所有训练都在分辨率为 224 的情况下进行。

| Dataset | Steps | Base LR |
|---|---|---|
| ImageNet | 20 000 | {0.003, 0.01, 0.03, 0.06} |
| CIFAR100 | 10 000 | {0.001, 0.003, 0.01, 0.03} |
| CIFAR10 | 10 000 | {0.001, 0.003, 0.01, 0.03} |
| Oxford-IIIT Pets | 500 | {0.001, 0.003, 0.01, 0.03} |
| Oxford Flowers-102 | 500 | {0.001, 0.003, 0.01, 0.03} |
| VTAB (19 tasks) | 2 500 | 0.01 |

表 4: 微调的超参数。所有模型均使用余弦学习率衰减、批量大小为 512、无权重衰减以及全局范数 1 的梯度裁剪进行微调。除非另有说明，微调分辨率为 384。

#### B.1.1 微调

我们使用带有 0.9 动量的 SGD 对所有 ViT 模型进行微调。我们在学习率上进行小范围的网格搜索，见表 4 中的学习率值域。为此，我们从训练集中使用小的子集（Pets 和 Flowers 为 10%，CIFAR 为 2%，ImageNet 为 1%）作为开发集，并在剩余数据上进行训练。对于最终结果，我们在整个训练集上训练并在相应的测试数据上评估。对于微调 ResNet 和混合模型，我们使用完全相同的设置，唯一的例外是 ImageNet，我们在学习率扫描中添加了另一个值 0.06。此外，对于 ResNet，我们还运行了 Kolesnikov et al. (2020) 的设置，并从这次运行和我们的扫描中选择最佳结果。最后，除非另有说明，所有微调实验均以 384 分辨率进行（与训练不同分辨率进行微调是常见做法 (Kolesnikov et al., 2020)）。

在将 ViT 模型转移到另一个数据集时，我们移除整个头部（两个线性层），并用一个零初始化的线性层替换它，该线性层输出目标数据集所需的类别数。我们发现这种方法比简单地重新初始化最后一层稍微更稳健一些。

对于 VTAB，我们遵循 Kolesnikov et al. (2020) 中的协议，并为所有任务使用相同的超参数设置。我们使用一个学习率为 0.01，并训练 2500 步（表 4）。我们通过在两个学习率和两种调度方案上进行小范围的扫描，并选择在 200 例验证集上获得最高 VTAB 分数的设置来确定此设置。除了不使用任务特定的输入分辨率外，我们遵循 Kolesnikov et al. (2020) 中使用的预处理方法。相反，我们发现 Vision Transformer 在所有任务中从高分辨率（$384 \times 384$）中获益最大。

#### B.1.2 自监督

我们采用*掩码块预测*目标进行初步的自监督实验。为此，我们将 50%的块嵌入进行破坏，方法是用可学习的[mask] 嵌入（占 80%）、随机选择其他块嵌入（占 10%）替换它们，或者保持原样（占 10%）。这个设置与 Devlin et al. (2019) 在语言任务中使用的设置非常相似。最后，我们使用每个被破坏块对应的表示来预测该块的 3-比特平均颜色（即总共 512 种颜色）。

我们使用批量大小为 4096 的 Adam 优化器在 JFT 上训练了我们的自监督模型 1M 步（约 14 个轮次），初始学习率为 $2 \cdot 10^{-4}$，热身 10k 步，并采用余弦学习率衰减。对于预测目标，我们尝试了以下情景：1）仅预测均值，3 比特颜色（即对 512 种颜色进行一次预测），2）并行预测 $4 \times 4$ 尺寸缩小版本的 $16 \times 16$ 补丁的 3 比特颜色（即对 512 种颜色进行 16 次预测），3）使用 L2 范数对完整补丁进行回归（即对 3 个 RGB 通道进行 256 次回归）。令人惊讶的是，我们发现所有方法都表现得相当好，尽管 L2 略逊一筹。由于选项 1）表现出最佳的 few-shot 性能，我们仅报告其最终结果。我们还尝试了与 Devlin et al. (2019) 相同的 15%损坏率，但在我们的 few-shot 指标上结果也稍差。

| | | ViT-B/16 | ViT-B/32 | ViT-L/16 | ViT-L/32 | ViT-H/14 |
|---|---|---|---|---|---|---|
| ImageNet | CIFAR-10 | 98.13 | 97.77 | 97.86 | 97.94 | - |
| | CIFAR-100 | 87.13 | 86.31 | 86.35 | 87.07 | - |
| | ImageNet | 77.91 | 73.38 | 76.53 | 71.16 | - |
| | ImageNet ReaL | 83.57 | 79.56 | 82.19 | 77.83 | - |
| | Oxford Flowers-102 | 89.49 | 85.43 | 89.66 | 86.36 | - |
| | Oxford-IIIT-Pets | 93.81 | 92.04 | 93.64 | 91.35 | - |
| ImageNet-21k | CIFAR-10 | 98.95 | 98.79 | 99.16 | 99.13 | 99.27 |
| | CIFAR-100 | 91.67 | 91.97 | 93.44 | 93.04 | 93.82 |
| | ImageNet | 83.97 | 81.28 | 85.15 | 80.99 | 85.13 |
| | ImageNet ReaL | 88.35 | 86.63 | 88.40 | 85.65 | 88.70 |
| | Oxford Flowers-102 | 99.38 | 99.11 | 99.61 | 99.19 | 99.51 |
| | Oxford-IIIT-Pets | 94.43 | 93.02 | 94.73 | 93.09 | 94.82 |
| JFT-300M | CIFAR-10 | 99.00 | 98.61 | 99.38 | 99.19 | 99.50 |
| | CIFAR-100 | 91.87 | 90.49 | 94.04 | 92.52 | 94.55 |
| | ImageNet | 84.15 | 80.73 | 87.12 | 84.37 | 88.04 |
| | ImageNet ReaL | 88.85 | 86.27 | 89.99 | 88.28 | 90.33 |
| | Oxford Flowers-102 | 99.56 | 99.27 | 99.56 | 99.45 | 99.68 |
| | Oxford-IIIT-Pets | 95.80 | 93.40 | 97.11 | 95.83 | 97.56 |

表 5: Top1 准确率 (%) of Vision Transformer 在不同数据集上当在 ImageNet 、ImageNet-21k 或 JFT300M 上预训练时。这些值对应于主文本中的图 3。模型在 384 分辨率下微调。注意，ImageNet 的结果是在不使用额外技术（Polyak 平均和 512 分辨率图像）的情况下计算的，这些技术用于实现表 2 中的结果。

最后，我们想指出，我们的带掩码的补丁预测方法不需要如此庞大的预训练量，也不需要像 JFT 这样的大型数据集，才能在 ImageNet 分类上获得类似的性能提升。也就是说，我们在经过 100k 次预训练步骤后观察到下游性能的收益递减，并且在 ImageNet 上进行预训练时也看到了类似的收益。

## C 附加结果

我们报告了与论文中给出的图表相对应的详细结果。表 5 对应于论文中的图 3，展示了在数据集大小不断增加（ImageNet、ImageNet-21k 和 JFT-300M）上进行预训练的不同 ViT 模型的迁移性能。表 6 对应于论文中的图 5，展示了不同大小的 ViT 、ResNet 和混合模型的迁移性能，以及它们预训练的估计计算成本。

## D 附加分析

### D.1 SGD 对比 Adam 在 ResNet 上的应用

ResNet 通常使用 SGD 进行训练，而我们使用 Adam 作为优化器的做法相当不寻常。这里我们展示了一些激发这一选择的实验结果。具体来说，我们将两个在 JFT 上预训练的 ResNet（50x1 和 152x2）的微调性能与 SGD 和 Adam 进行了比较。对于 SGD，我们采用了 Kolesnikov et al. (2020) 推荐的超参数。结果见表 7。Adam 预训练在大多数数据集上的表现优于 SGD 预训练，并且平均表现也更好。这证明了将 Adam 作为优化器用于在 JFT 上预训练 ResNet

| name | Epochs | ImageNet | ImageNet ReaL | CIFAR-10 | CIFAR-100 | Pets | Flowers | exaFLOPs |
|---|---|---|---|---|---|---|---|---|
| ViT-B/32 | 7 | 80.73 | 86.27 | 98.61 | 90.49 | 93.40 | 99.27 | 55 |
| ViT-B/16 | 7 | 84.15 | 88.85 | 99.00 | 91.87 | 95.80 | 99.56 | 224 |
| ViT-L/32 | 7 | 84.37 | 88.28 | 99.19 | 92.52 | 95.83 | 99.45 | 196 |
| ViT-L/16 | 7 | 86.30 | 89.43 | 99.38 | 93.46 | 96.81 | 99.66 | 783 |
| ViT-L/16 | 14 | 87.12 | 89.99 | 99.38 | 94.04 | 97.11 | 99.56 | 1567 |
| ViT-H/14 | 14 | 88.08 | 90.36 | 99.50 | 94.71 | 97.11 | 99.71 | 4262 |
| ResNet50x1 | 7 | 77.54 | 84.56 | 97.67 | 86.07 | 91.11 | 94.26 | 50 |
| ResNet50x2 | 7 | 82.12 | 87.94 | 98.29 | 89.20 | 93.43 | 97.02 | 199 |
| ResNet101x1 | 7 | 80.67 | 87.07 | 98.48 | 89.17 | 94.08 | 95.95 | 96 |
| ResNet152x1 | 7 | 81.88 | 87.96 | 98.82 | 90.22 | 94.17 | 96.94 | 141 |
| ResNet152x2 | 7 | 84.97 | 89.69 | 99.06 | 92.05 | 95.37 | 98.62 | 563 |
| ResNet152x2 | 14 | 85.56 | 89.89 | 99.24 | 91.92 | 95.75 | 98.75 | 1126 |
| ResNet200x3 | 14 | 87.22 | 90.15 | 99.34 | 93.53 | 96.32 | 99.04 | 3306 |
| R50x1+ViT-B/32 | 7 | 84.90 | 89.15 | 99.01 | 92.24 | 95.75 | 99.46 | 106 |
| R50x1+ViT-B/16 | 7 | 85.58 | 89.65 | 99.14 | 92.63 | 96.65 | 99.40 | 274 |
| R50x1+ViT-L/32 | 7 | 85.68 | 89.04 | 99.24 | 92.93 | 96.97 | 99.43 | 246 |
| R50x1+ViT-L/16 | 7 | 86.60 | 89.72 | 99.18 | 93.64 | 97.03 | 99.40 | 859 |
| R50x1+ViT-L/16 | 14 | 87.12 | 89.76 | 99.31 | 93.89 | 97.36 | 99.11 | 1668 |

表 6: 模型缩放实验的详细结果。这些对应于主论文中的图 5。我们展示了在几个数据集上的迁移准确率以及预训练计算量（以 exaFLOPs 为单位）。

| | ResNet50 | | ResNet152x2 | |
|---|---|---|---|---|
| Dataset | Adam | SGD | Adam | SGD |
| ImageNet | 77.54 | 78.24 | 84.97 | 84.37 |
| CIFAR10 | 97.67 | 97.46 | 99.06 | 99.07 |
| CIFAR100 | 86.07 | 85.17 | 92.05 | 91.06 |
| Oxford-IIIT Pets | 91.11 | 91.00 | 95.37 | 94.79 |
| Oxford Flowers-102 | 94.26 | 92.06 | 98.62 | 99.32 |
| Average | 89.33 | 88.79 | 94.01 | 93.72 |

表 7: 微调用 Adam 和 SGD 预训练的 ResNet 模型。

的选择是合理的。请注意，绝对数值比 Kolesnikov et al. (2020) 报告的要低，因为我们只预训练了 7 轮次，而不是 30 轮次。

### D.2 变换形状

我们对缩放 Transformer 架构的不同维度进行了消融实验，以找出哪些维度最适合用于扩展到非常大的模型。图 8 显示了在不同配置下 ImageNet 上的 5-shot 性能。所有配置均基于一个 ViT 模型，具有 8 层，$D = 1024$ ，$D_{MLP} = 2048$ 并且 patch 大小为 32 ，这是所有线条的交点。我们可以看到，增加深度带来了最大的改进，这些改进在 64 层之前非常明显。然而，在 16 层之后已经可以看到回报递减的现象。有趣的是，增加网络宽度似乎导致的变化最小。减小 patch 大小并因此增加有效的序列长度显示出了令人惊讶的稳健改进，而没有引入额外的参数。这些发现表明，计算能力可能是比参数数量更好的性能预测器，并且如果需要缩放的话，应该更强调深度而不是宽度。总体而言，我们发现按比例缩放所有维度可以带来稳健的改进。

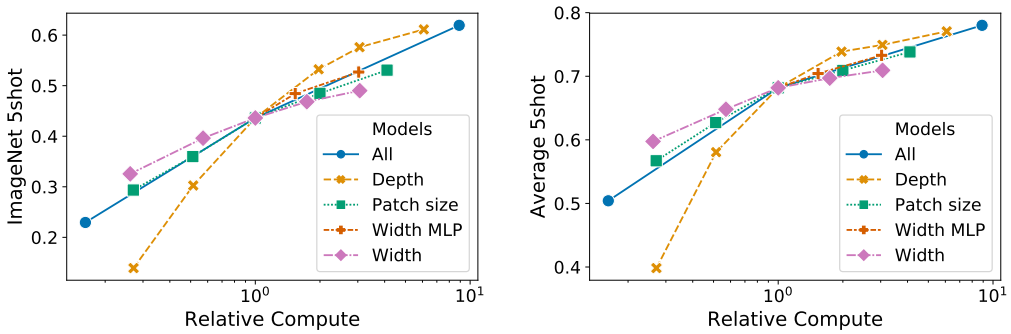



图 8: 缩放 Vision Transformer 的不同模型维度。

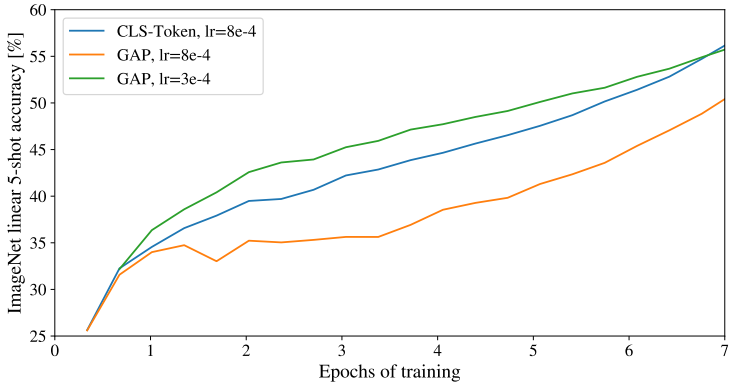



图 9: 类-token 和全局平均池化分类器的比较。两者表现相似，但需要不同的学习率。

### D.3 头部类型和 class 词元

为了尽量接近原始的 Transformer 模型，我们使用了一个额外的 [class] 词元，该词元被视为图像表示。然后通过一个小的多层感知机（MLP）将这个词元的输出转换为类别预测，在单个隐藏层中使用 tanh 作为非线性函数。

这一设计继承自用于文本的 Transformer 模型，并在主论文中广泛使用。最初的尝试仅使用图像块嵌入（image-patch embeddings），对其进行全局平均汇聚（GAP），然后接一个线性分类器——就像 ResNet 的最后一层特征图一样——表现非常糟糕。然而，我们发现这既不是由于额外的词元，也不是由于 GAP 操作。相反，性能差异完全可以由对不同的学习率的需求来解释，请参见图 9。

### D.4 位置嵌入

我们通过使用位置嵌入对编码空间信息的不同方式进行了一些实验。我们尝试了以下情况：

- 提供无位置信息：将输入视为一个 补丁的集合。
- 一维位置嵌入：将输入视为光栅顺序（本文其他实验的默认设置）的一系列补丁序列。

| Pos. Emb. | Default/Stem | Every Layer | Every Layer-Shared |
|---|---|---|---|
| No Pos. Emb. | 0.61382 | N/A | N/A |
| 1-D Pos. Emb. | 0.64206 | 0.63964 | 0.64292 |
| 2-D Pos. Emb. | 0.64001 | 0.64046 | 0.64022 |
| Rel. Pos. Emb. | 0.64032 | N/A | N/A |

表 8: 在 ImageNet 5-shot 线性设置下，使用 ViT-B/16 模型对位置嵌入进行消融研究的结果。

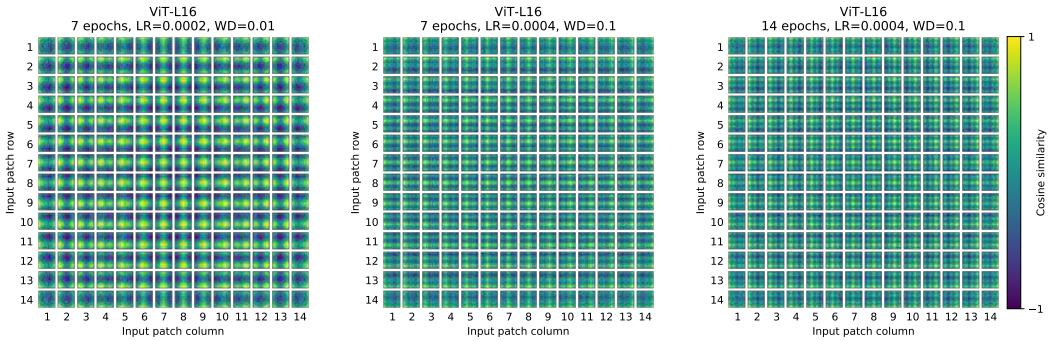



图 10: 使用不同超参数训练的模型的位置嵌入。

- 2 维位置嵌入：将输入视为二维的补丁网格。在这种情况下，学习两组嵌入，每组对应一个轴，$X$-嵌入和 $Y$-嵌入，每组的大小为 $D/2$。然后，根据输入中路径上的坐标，我们将 $X$ 和 $Y$ 嵌入进行连结，以获得该补丁的最终位置嵌入。

- 相对位置嵌入：考虑补丁之间的相对距离以编码空间信息，而不是它们的绝对位置。为此，我们使用一维相对注意力，在其中定义所有可能补丁对之间的相对距离。因此，对于每个给定的补丁对（一个作为查询，另一个作为注意力机制中的键/值），我们有一个偏移量 $p_q - p_k$ ，其中每个偏移量都与一个嵌入相关联。然后，我们简单地运行额外的注意力，其中我们使用原始查询（查询的内容），但使用相对位置嵌入作为键。接着，我们将相对注意力的 logits 作为偏差项，并在应用 Softmax 之前将其添加到主注意力（基于内容的注意力）的 logits 中。

除了对空间信息进行编码的不同方式外，我们还尝试了将这些信息纳入模型的不同方法。对于一维和二维的位置嵌入，我们尝试了三种不同的情况：（1）在模型的主干之后、将输入馈送到 Transformer 编码器之前，添加位置嵌入（本文其他实验的默认设置）；（2）在每一层开始时学习并添加位置嵌入；（3）在每一层开始时添加共享的学成位置嵌入。

表 8 概述了在这个消融研究中的结果，研究对象是 ViT-B/16 模型。正如我们可以看到的，没有位置嵌入的模型和有位置嵌入的模型在性能上存在很大差距，但在编码位置信息的不同方式之间几乎没有差异。我们推测，由于我们的 Transformer 编码器在 Patch 级别的输入上运行，而不是像素级别，因此在如何编码空间信息方面存在的差异并不重要。更确切地说，在 Patch 级别的输入中，空间维度比原始像素级输入小得多，例如，$14 \times 14$ 相比于 $224 \times 224$，并且以这种分辨率学习表示这些空间关系对这些不同的位置编码策略来说同样容易。即便如此，网络学习到的位置嵌入相似性的具体模式仍然取决于训练超参数（图 10）。

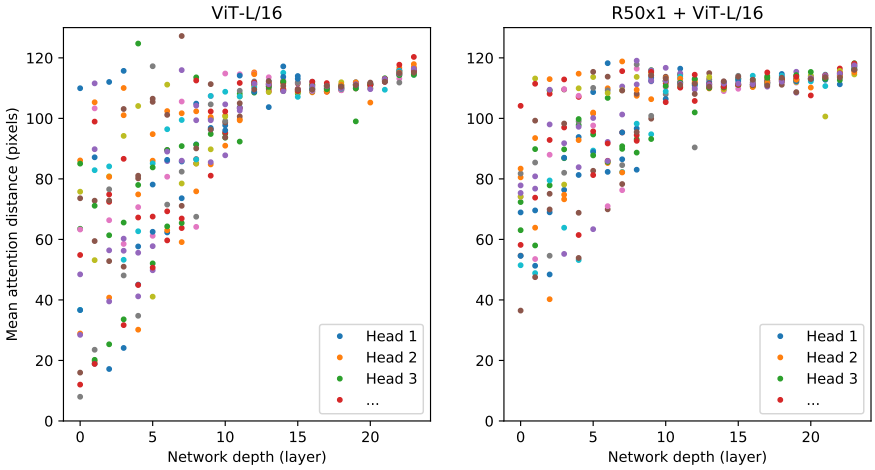



图 11: 头和网络深度关注区域的大小。通过将查询像素与所有其他像素之间的距离平均计算注意力距离，该距离由注意力权重加权。每个点显示一层中 16 个头之一在图像上的平均注意力距离。图像宽度为 224 像素。

### D.5 经验计算成本

我们还对实际硬件上架构的实时性能感兴趣，由于通道宽度和缓存大小等细节，理论浮点运算次数（FLOPs）并不总能很好地预测这一点。为此，我们在 TPUv3 加速器上对主要感兴趣的模型进行了推理速度的计时测试；推理速度和反向传播速度之间的差异是一个恒定的、与模型无关的因素。

图 12（左）显示了每个核心每秒可以处理多少张图像，涵盖了各种输入大小。每个点代表在广泛的批量大小范围内测得的峰值性能。如图所示，ViT 的理论双二次缩放与图像大小的关系仅在最大模型和最大分辨率下才开始显现。

另一个感兴趣的量是每个模型可以 fitting 到一个内核上的最大批量大小，越大越好对于缩放到大数据集。图 12（右）显示了相同模型集的这个量。这表明大型 ViT 模型在内存效率方面相对于 ResNet 模型具有明显优势。

### D.6 轴向注意力

轴向自注意力 (Huang et al., 2020; Ho et al., 2019) 是一种简单而有效的方法，可以在组织为多维张量的大输入上运行自注意力。轴向自注意力的一般思想是沿着输入张量的单个轴执行多次注意力操作，而不是对输入的展平版本应用一维注意力。在轴向自注意力中，每次注意力沿特定轴混合信息，同时保持其他轴上的信息独立。沿着这个思路，Wang et al. (2020b) 提出了轴向 ResNet 模型，其中 ResNet50 中的所有卷积核大小为 $3 \times 3$ 的卷积都被轴向自注意力（即行和列注意力，并辅以相对位置编码）所取代。我们已经将轴向 ResNet 实现为一个基准模型。[3].

[3]我们的实现基于 https://github.com/csrhddlam/axial-deeplab 中的开源 PyTorch 实现。在我们的实验中，我们在准确率方面重现了 (Wang et al., 2020b) 报告的得分，然而，与开源实现类似，我们的实现在 TPU 上非常慢。因此，我们无法用它进行大规模的广泛实验。这些可能通过精心优化的实现来解锁。

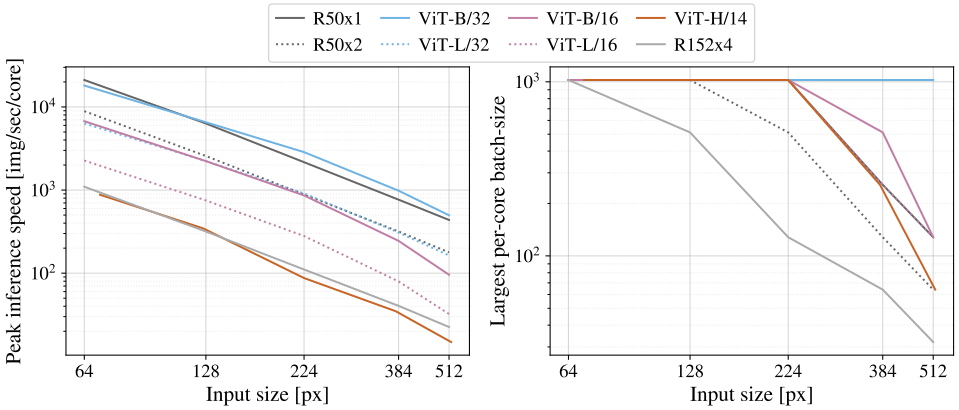



图 12: **左**: 不同架构在各种输入大小下的真实时钟时间。ViT 模型的速度与类似的 ResNet 相当。**右**: 在不同输入大小下，各种架构在设备上的每核最大批量大小。ViT 模型显然更节省内存。

而且，我们对 ViT 进行了修改，使其能够处理二维形状的输入，而不是一维的补丁序列，并结合了轴向 Transformer 块，其中不再是自注意力后跟一个 MLP，而是行自注意力加 MLP，然后是列自注意力加 MLP。

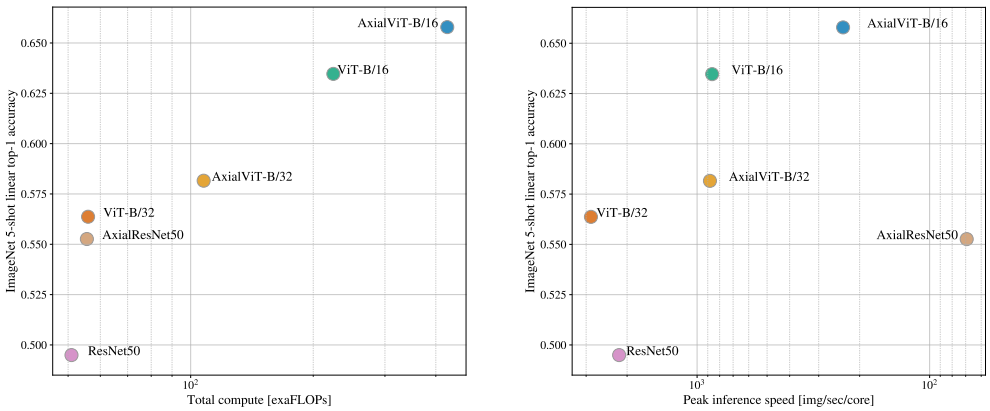



图 13: Axial-注意力模型在 ImageNet 5-shot 线性上的 top-1 准确率与它们在 FLOPs 数量（**左**）和推理时间（**左**）方面的速度对比。

图 13 展示了在 ImageNet 5-shot 线性评估中，Axial ResNet、Axial-ViT-B/32 和 Axial-ViT-B/16 的性能，这些模型都在 JFT 数据集上进行了预训练，与预训练计算量（以 FLOPs 数量和推理时间为例）进行对比。如我们所见，无论是 Axial-ViT-B/32 还是 Axial-ViT-B/16，在性能方面都优于其对应的 ViT-B 模型，但这也带来了更高的计算成本。这是因为 Axial-ViT 模型中的每个带有全局自注意力的 Transformer 块被替换为两个轴向 Transformer 块，分别处理行和列的自注意力，并且尽管轴向情况下自注意力操作的序列长度更小，但每个 Axial-ViT 块都会额外增加一个 MLP。对于 AxialResNet，虽然在准确率/计算量权衡方面看起来合理（图 13，左），但在 TPU 上的实现速度非常慢（图 13，右）。

### D.7 注意力距离

为了理解 ViT 如何利用自注意力在整个图像中整合信息（图 11），我们分析了不同层上注意力权重的平均跨度距离。这个“注意力距离”类似于 CNN 中的感受野大小。在较低层中，注意力距离在各个头之间变化很大，一些头关注整个图像，而另一些头则关注于查询位置附近的小区域。随着深度增加，所有头的注意力距离都增加。在网络的后半部分，大多数头广泛地关注于不同的词元。

### D.8 注意力图

为了计算从输出词元到输入空间的注意力映射（图 6 和 14），我们使用了 Attention Rollout (Abnar & Zuidema, 2020)。简而言之，我们对所有头的 ViT-L/16 的注意力权重取平均值，然后递归地将所有层的权重矩阵相乘。这解释了通过所有层的词元间注意力的混合。

### D.9 ObjectNet 结果

我们还在 ObjectNet 基准上评估了我们的旗舰 ViT-H/14 模型，采用 Kolesnikov et al. (2020) 中的评估设置，得到了 82.1%的 top-5 准确率和 61.7%的 top-1 准确率。

### D.10 VTAB 分解

表 9 显示了在每个 VTAB-1k 任务中获得的得分。

表 9: VTAB-1k 在各个任务上的性能分解。

| | Caltech101 | CIFAR-100 | DTD | Flowers102 | Pets | Sun397 | SVHN | Camelyon | EuroSAT | Resisc45 | Retinopathy | Clevr-Count | Clevr-Dist | DMLab | dSpr-Loc | dSpr-Ori | KITTI-Dist | sNORB-Azim | sNORB-Elev | Mean |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ViT-H/14 (JFT) | 95.3 | 85.5 | 75.2 | 99.7 | 97.2 | 65.0 | 88.9 | 83.3 | 96.7 | 91.4 | 76.6 | 91.7 | 63.8 | 53.1 | 79.4 | 63.3 | 84.5 | 33.2 | 51.2 | 77.6 |
| ViT-L/16 (JFT) | 95.4 | 81.9 | 74.3 | 99.7 | 96.7 | 63.5 | 87.4 | 83.6 | 96.5 | 89.7 | 77.1 | 86.4 | 63.1 | 49.7 | 74.5 | 60.5 | 82.2 | 36.2 | 51.1 | 76.3 |
| ViT-L/16 (I21k) | 90.8 | 84.1 | 74.1 | 99.3 | 92.7 | 61.0 | 80.9 | 82.5 | 95.6 | 85.2 | 75.3 | 70.3 | 56.1 | 41.9 | 74.7 | 64.9 | 79.9 | 30.5 | 41.7 | 72.7 |

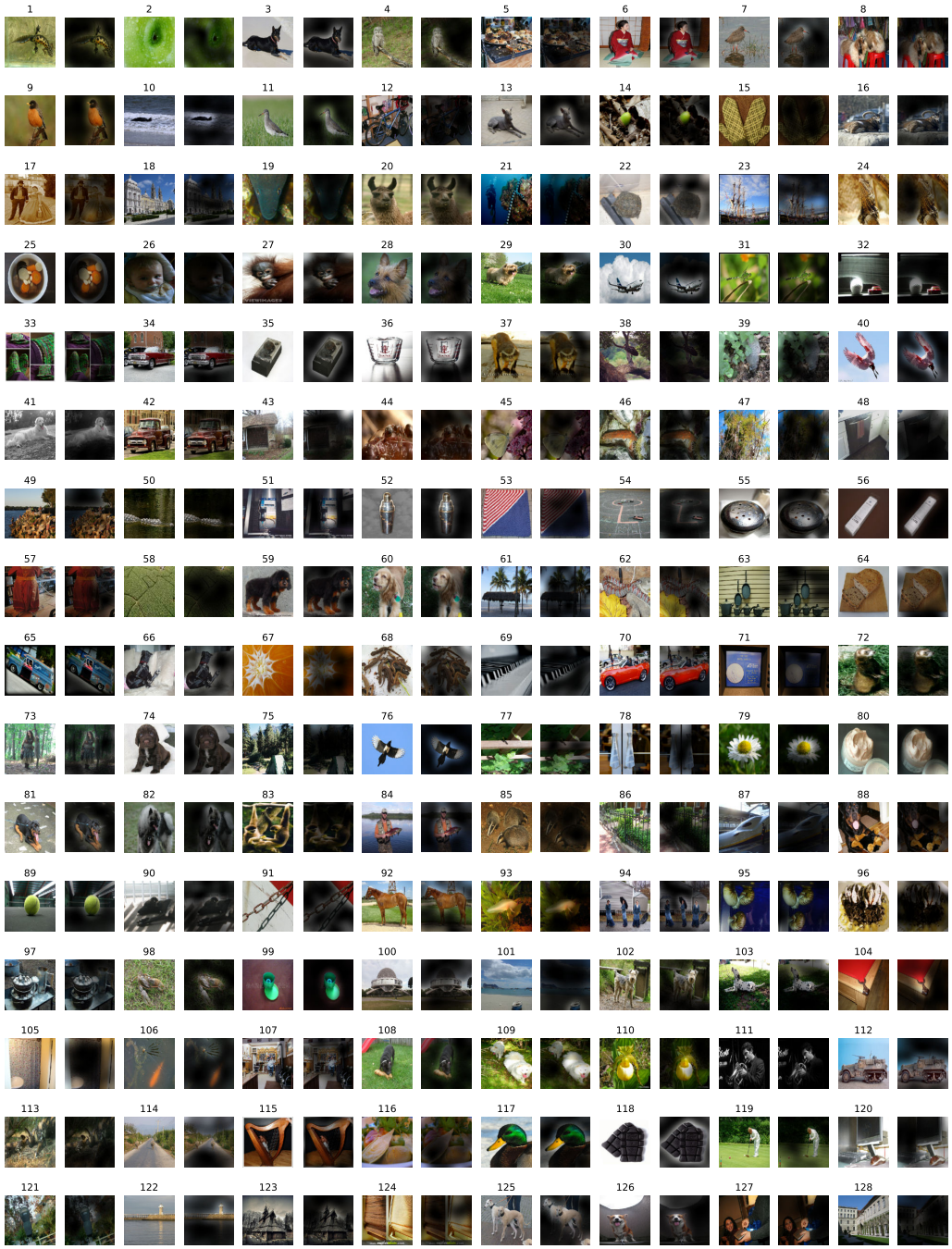


图 14: 进一步的例子如图 6 所示（随机选择）。